

Table des matières

Cours 01 — Paysage SQL et modèle relationnel.....	1
Objectifs.....	1
Théorie.....	2
Et maintenant : la pratique.....	13
Points clés à retenir.....	13
Exercices.....	14

Cours 01 — Paysage SQL et modèle relationnel

Module 1/12 — Formation SQL avancé

Comment lire ce module — On part de zéro : ce qu'est une base de données, les mots du métier, les familles de commandes. On travaille sur une vraie base déjà en place au Cours 00, *AdventureWorks DW*, un entrepôt rangé en étoile, qui servira d'exemple à toute la formation. On finit par trois choses que même des gens expérimentés font de travers : l'ordre d'exécution, les jointures et NULL. Ce fichier pose la théorie ; le laboratoire (Labo_01.md) vous fait tout pratiquer, car le SQL s'apprend avec les doigts, pas juste avec les yeux.

Objectifs

À la fin de ce module, vous saurez :

- expliquer avec vos mots ce que sont data, database, DBMS, RDBMS et SQL ;
- reconnaître les cinq familles de commandes SQL (DDL, DML, DQL, DCL, TCL), dire à quoi sert chacune, et laquelle peut s'annuler ;
- comprendre pourquoi le même SQL se comporte un peu différemment d'un moteur à l'autre ;
- vous repérer dans AdventureWorks DW, l'entrepôt en étoile qui nous accompagnera tout du long, et situer l'étoile parmi les autres façons de modéliser ;
- lire une requête dans l'ordre où le moteur l'exécute, ce qui explique la plupart des erreurs de débutant ;
- relier une table de faits à ses dimensions avec une jointure, et éviter le piège classique du LEFT JOIN ;
- travailler avec NULL, la valeur « on ne sait pas », sans vous faire piéger, et reconnaître une absence déguisée en valeur, comme le « NA » de notre entrepôt.

Théorie

1. Cinq mots à connaître : data, database, DBMS, RDBMS, SQL

Vous avez déjà utilisé un tableur comme Excel : des lignes, des colonnes, des feuilles. Une base de données, c'est la même idée, mais pensée pour stocker beaucoup plus, à plusieurs, sans abîmer les données. Cinq mots reviennent sans cesse dans ce monde-là. On les confond souvent. Voici ce que chacun veut dire.

Mot	Ce que c'est	Exemple
Data	les données, les faits bruts. Seuls, ils ne veulent rien dire	1049.00, "Mountain Bike", une date
Database	des données rangées pour qu'on les retrouve vite. C'est le contenu, pas le logiciel	des tables reliées entre elles
DBMS	le logiciel qui gère la base : il range, répond, protège, sauvegarde	tables ou non : MongoDB (documents), Redis (clé-valeur)
RDBMS	un DBMS qui range tout en tables reliées par des clés. Le type le plus courant	DuckDB, PostgreSQL, MySQL, Oracle, SQL Server, Snowflake
SQL	le langage, les phrases qu'on écrit pour parler au RDBMS	SELECT * FROM dimproduct

Les cinq s'emboîtent, du plus concret au plus abstrait : la *data* compose une *database*, gérée par un *DBMS*, dont la version en tables est le *RDBMS*, à qui on parle en *SQL*.

Deux pièges de vocabulaire, à écarter tout de suite.

SQL n'est pas le moteur. SQL est le langage, DuckDB est le logiciel qui l'exécute. On peut parler SQL à plusieurs moteurs différents, et c'est tout l'intérêt d'apprendre le langage plutôt qu'un produit.

Un tableur n'est pas un RDBMS. Excel a bien des lignes et des colonnes, mais il lui manque les garde-fous d'une vraie base : les clés, les liens garantis entre tables, et la gestion de plusieurs personnes qui écrivent en même temps.

2. Les cinq familles de commandes SQL

Labo : Labo_01.md, partie 1.

Toutes les commandes SQL se rangent en cinq familles, selon ce qu'elles font. C'est la carte du territoire : une fois que vous l'avez en tête, vous savez toujours dans quelle catégorie se trouve ce que vous faites.

Famille	Sigle	À quoi ça sert	Commandes clés	Annulable ?
<i>Data Definition Language</i>	DDL	créer ou modifier la structure (les tables)	CREATE, ALTER, DROP, TRUNCATE, RENAME	selon le moteur : oui sur nos deux terrains, non ailleurs
<i>Data Manipulation Language</i>	DML	ajouter ou changer les données	INSERT, UPDATE, DELETE	oui, tant qu'on n'a pas validé
<i>Data Query Language</i>	DQL	lire les données	SELECT	sans objet, lecture seule
<i>Data Control Language</i>	DCL	gérer les droits	GRANT, REVOKE	selon le moteur, quand il en a un
<i>Transaction Control Language</i>	TCL	valider ou annuler un lot de changements	COMMIT, ROLLBACK, SAVEPOINT	sans objet, c'est l'outil qui annule

Une façon simple de s'en souvenir, avec l'image du classeur. Le DDL fabrique le contenant, le tiroir et les fiches vierges. Le DML s'occupe du contenu : on ajoute, corrige ou retire des fiches. Le DQL pose les questions, « montre-moi les fiches qui... ». Le DCL distribue les clés : qui a le droit de lire, qui a le droit d'écrire. Le TCL est le filet de sécurité : on valide un paquet de changements d'un coup, ou on annule tout si quelque chose a mal tourné.

La colonne « Annulable » mérite un mot. Quand vous modifiez des données (DML), le changement n'est pas gravé dans le marbre tout de suite : tant que vous n'avez pas fait COMMIT, vous pouvez tout annuler avec ROLLBACK. Pour les commandes de structure (DDL), la réponse dépend du moteur, et nos deux terrains sont du même côté : sur DuckDB comme sur PostgreSQL, un CREATE TABLE, un TRUNCATE ou même un DROP TABLE lancés dans une transaction disparaissent au ROLLBACK, table et contenu revenus comme avant. D'autres moteurs valident le DDL d'office, en auto-commit, et une commande lancée ne se reprend pas : c'est le cas de Snowflake, d'Oracle et de MySQL, d'après leur documentation. D'où la prudence de rigueur avec DROP, partout : le filet dépend du moteur sur lequel vous êtes, et vous ne travaillerez pas toute votre carrière sur celui-ci. On revient sur ce qui s'annule au Cours 03, section 8, et sur les transactions elles-mêmes au Cours 11.

Les cinq familles, sur une petite table jetable :

-- DDL : je cree la structure (une table vide, juste pour l'exemple)

CREATE TABLE demo (**id** INTEGER, nom TEXT);

-- DML : j'ajoute, puis je corrige une donnee

INSERT INTO demo **VALUES** (1, 'Sophie');

UPDATE demo **SET** nom = 'Sophie Tremblay' **WHERE id** = 1;

-- DQL : je lis

SELECT * **FROM** demo;

-- DCL : je donne le droit de lecture a quelqu'un d'autre

```
GRANT SELECT ON demo TO analyste;
```

```
-- TCL : je valide tout ce qui precede (ou j'annule avec ROLLBACK)  
COMMIT;
```

Portabilité — Les cinq familles sont la carte du langage, pas celle d'un moteur donné, et les deux dernières lignes ci-dessus ne passent pas sur DuckDB. Le GRANT échoue sur une erreur de syntaxe, parce que DuckDB n'a pas de DCL du tout : c'est un moteur intégré, qui tourne dans votre session et n'a personne à qui donner des droits ; les droits sont ceux du fichier, réglés par le système d'exploitation. Le COMMIT échoue sur `cannot commit - no transaction is active`, parce qu'aucun BEGIN TRANSACTION ne l'a précédé et que chaque commande a donc déjà été validée seule. Les deux gestes existent sur PostgreSQL, qui a des rôles et des transactions explicites, et c'est là que vous les pratiquerez, aux Cours 03 et 11 ; le GRANT y demande seulement que le rôle ait été créé d'abord, faute de quoi il répond que le rôle n'existe pas.

Vous en avez déjà fait. Au Cours 00, en exécutant `monter_entrepot_dw.sql`, vous avez utilisé du DDL (CREATE TABLE) pour bâtir les tables de l'entrepôt ; puis, dans le même fichier, du DML de remplissage (COPY). Ce cours-ci se concentre sur le DQL : les interroger.

Bon à savoir — Certains rangent le SELECT (DQL) dans le DML, et comptent donc quatre familles. La version à cinq est la plus répandue et la plus claire, et c'est celle qu'on attend en entretien.

3. Un langage, deux moteurs

Labo : Labo_01.md, partie 3.

SQL est un langage normalisé : il existe une norme officielle (ANSI, puis ISO) que tout le monde suit. Les bases du SQL sont donc les mêmes partout ; seuls les détails changent d'un moteur à l'autre. C'est comme l'anglais : même grammaire à Londres et à New York, quelques mots qui diffèrent.

Cette formation est analytique d'abord, et elle s'appuie sur deux moteurs qui représentent chacun un monde.

DuckDB est un moteur analytique (OLAP) : il est fait pour balayer beaucoup de lignes d'un coup et en tirer des totaux, au service de tableaux de bord, de rapports et de data science. C'est le moteur de référence de la formation, celui sur lequel la théorie s'illustre et sur lequel tourne l'entrepôt monté au Cours 00. Il est gratuit et local, et il n'a rien d'un serveur : c'est un moteur intégré, un seul programme qui ouvre un fichier, comme SQLite, mais rangé par colonnes plutôt que par lignes, ce qui est exactement ce qu'il faut pour l'analyse.

PostgreSQL est un moteur transactionnel (OLTP) : il fait tourner des applications où l'on lit et écrit sans cesse de petites quantités de données. Il est gratuit et local lui aussi, et c'est un serveur, un programme qui tourne en continu et à qui plusieurs clients se connectent. Ce n'est pas un simple figurant : à partir du Cours 02, chaque module a deux laboratoires, un sur chaque terrain, qui posent les mêmes questions sur deux schémas opposés.

En clair, on apprend un seul langage et on le pratique sur deux terrains. Ce qui change d'un monde à l'autre n'est presque jamais la syntaxe, c'est la forme du schéma, et c'est précisément ce que le double laboratoire vous fait sentir. Quand une commande diffère entre les deux, ou entre eux et une grande plateforme, un encadré « Portabilité » vous le signale.

Reste une famille de moteurs qu'on ne pratiquera pas ici, et qu'il faut situer : les entrepôts de données dans le cloud, comme Snowflake, BigQuery ou Redshift. Ils font le même métier que DuckDB, l'analyse de gros volumes, mais sur des machines louées à l'heure, sans rien à installer et sans limite de taille. Le langage y

est le même à peu de chose près, et c’est pourquoi les encadrés « Portabilité » de cette formation nomment souvent Snowflake : c’est la plateforme que vous rencontrerez le plus probablement en entreprise, et elle fait l’objet d’une formation à part dans ce parcours. Ce que ces encadrés en disent est tiré de sa documentation, pas d’une exécution : ce cours ne lance que ce qui tourne sur vos deux moteurs.

Portabilité, deux différences à garder en tête — La première est que chaque moteur ajoute ses raccourcis. QUALIFY, qu’on verra au Cours 04, tourne sur DuckDB, manque à PostgreSQL, et existe aussi sur Snowflake d’après sa documentation. La seconde touche la casse des noms de colonnes non mis entre guillemets : DuckDB garde la casse que vous avez écrite à la déclaration et compare les noms sans en tenir compte, PostgreSQL replie tout en minuscules, et Snowflake force les majuscules en interne, d’après sa documentation. C’est un détail qui surprend au passage de l’un à l’autre, et la section 5 y revient.

4. Le modèle relationnel, sans jargon

Labo : Labo_01.md, partie 1.

« Relationnel » fait savant, mais l’idée est simple : on range les données en tables, et on relie les tables entre elles. Trois notions suffisent pour démarrer.

La **clé primaire** est une colonne qui identifie chaque ligne sans ambiguïté, un numéro unique, jamais vide. Dans la table DimProduct, c’est ProductKey. Deux produits ne devraient pas avoir le même. Dans un entrepôt, cette clé est souvent un numéro de substitution, un identifiant que l’entrepôt attribue lui-même, indépendant des codes du système d’origine.

La **clé étrangère** est une colonne qui pointe vers la clé primaire d’une autre table. Chaque vente de la table de faits FactInternetSales porte un ProductKey qui renvoie à DimProduct : on sait ainsi quel produit a été vendu. C’est ce lien qui rendra les jointures possibles (section 7).

Qui tient ces promesses, et où. La question est plus importante qu’elle n’en a l’air, parce que la réponse ne dépend pas seulement du moteur : elle dépend aussi de ce que le chargement a pris la peine de déclarer.

Cas	Ce qu’il advient des clés
Le transactionnel, PostgreSQL	les clés sont déclarées et le moteur les fait respecter. Un doublon de clé primaire est refusé, et une vente qui désigne un produit inexistant aussi. La promesse est tenue à chaque écriture
Notre entrepôt, sur DuckDB	aucune clé n’est déclarée. Le script du Cours 00 crée les huit tables sans clé primaire ni clé étrangère, et rien n’empêche donc un chargement d’écrire deux fois la même
Un entrepôt cloud, comme Snowflake	les clés se déclarent et s’affichent, mais le moteur ne les vérifie pas, d’après sa documentation. Elles disent l’intention au lecteur et aux outils, sans rien garantir

La ligne du milieu mérite une précision, parce qu’elle est facile à mal lire. Ce n’est pas que DuckDB serait incapable de tenir une clé : si on lui en déclare une, il la fait respecter, comme PostgreSQL, et le Labo 03 vous le fera constater en essayant d’insérer un doublon. C’est notre montage qui n’en déclare aucune, ce qui est

le choix ordinaire d'un entrepôt : les données y arrivent par lots contrôlés en amont, et vérifier chaque ligne à l'écriture coûterait du temps sans rien apprendre. La conséquence, elle, est bien réelle.

Piège — une clé non déclarée n'est pas une clé tenue, et une clé déclarée ne l'est pas toujours non plus. Dans notre entrepôt, le `ProductKey` de `DimProduct` est bien unique, 606 valeurs pour 606 lignes, mais parce que le chargement s'est bien passé, pas parce qu'une règle l'impose. Sur un entrepôt, l'unicité d'une clé se contrôle, elle ne se suppose pas :

```
SELECT COUNT(*) AS lignes, COUNT(DISTINCT ProductKey) AS cles FROM dbo.DimProduct;
```

Deux comptes égaux, la clé tient. C'est ce genre de contrôle qu'un projet d'ingénierie analytique transforme en test automatique, et c'est le travail du Cours 12.

Le **schéma**, au sens « tiroir », regroupe les tables qui vont ensemble. Notre entrepôt range tout dans un seul schéma, `dbo` : on écrit `dbo.DimProduct`, c'est-à-dire schéma point table.

Une dernière chose, qui surprend souvent : une table n'a pas d'ordre. Tant que vous n'écrivez pas `ORDER BY`, le moteur peut vous renvoyer les lignes dans l'ordre qu'il veut, et cet ordre peut changer d'une fois à l'autre. Si l'ordre compte, dites-le avec `ORDER BY`.

5. La base d'exemple : AdventureWorks DW, un entrepôt en étoile

Labo : Labo_01.md, partie 1.

Pour bien apprendre, rien ne vaut une vraie base. On utilise AdventureWorks, la base d'exemple la plus connue, publiée par Microsoft. Elle décrit une entreprise fictive, Adventure Works Cycles, qui fabrique et vend des vélos partout dans le monde.

On travaille sur sa version entrepôt de données (*Data Warehouse*, DW), rangée en schéma en étoile. Au centre, une grande table de faits ; tout autour, des tables de dimensions.

Table	Rôle	Colonnes qu'on utilisera
FactInternetSales	faits : une ligne égale une ligne de commande en ligne, pas une commande	ProductKey, OrderQuantity, SalesAmount, OrderDateKey
DimProduct	dimension produit	ProductKey (clé), EnglishProductName, Color (« NA » quand elle ne s'applique pas), ListPrice (peut être vide), ProductSubcategoryKey (peut être vide)
DimProductSubcategory	dimension sous-catégorie	ProductSubcategoryKey (clé), ProductCategoryKey, EnglishProductSubcategoryName
DimProductCategory	dimension catégorie	ProductCategoryKey (clé), EnglishProductCategoryName

Comment ces tables se relient. La table de faits est au centre, et chaque dimension s’y raccroche par une clé. La chaîne produit, elle, est une petite hiérarchie qui se déroule en s’éloignant du centre : FactInternetSales → DimProduct → DimProductSubcategory → DimProductCategory. Le tableau ci-dessous dit chaque lien dans le sens où on l’écrit dans un JOIN.

De cette table	Vers cette table	Par cette clé	Ce que le lien donne
FactInternetSales	DimProduct	ProductKey	le nom, la couleur et le prix du produit vendu
DimProduct	DimProductSubcategory	ProductSubcategoryKey	la sous-catégorie du produit, quand il en a une
DimProductSubcategory	DimProductCategory	ProductCategoryKey	la catégorie qui chapeaute la sous-catégorie
FactInternetSales	DimDate	OrderDateKey	l’année, le mois, le trimestre de la vente

L’étoile n’est pas la seule façon de modéliser un entrepôt, et il vaut la peine de savoir ce qui existe à côté, ne serait-ce que pour comprendre pourquoi on a choisi celle-là.

Modèle	Le principe	Ce qu’il gagne	Ce qu’il coûte
Étoile (ce cours)	un fait au centre, des dimensions larges et aplaties autour	des requêtes courtes, peu de jointures, un vocabulaire proche du métier	de la redondance dans les dimensions, des mises à jour lourdes
Flocon	les dimensions sont elles-mêmes normalisées en sous-tables	moins de redondance, une hiérarchie explicite	une jointure de plus par niveau, des requêtes plus longues
Normalisé (3NF)	le modèle transactionnel, appliqué à l’entrepôt	intégrité maximale, aucune donnée en double	des requêtes analytiques très longues, difficile pour les outils de BI
Table large (<i>one big table</i>)	tout aplati dans une seule table, sans aucune jointure	la lecture la plus rapide possible, trivial à consommer	ingérable dès qu’il y a plusieurs faits, redondance extrême
Data Vault	des hubs, des liens et des satellites, pensés pour l’historisation	souplesse et traçabilité face aux changements de source	complexité forte, jamais exposé tel quel aux analystes

En pratique, l’étoile domine parce qu’elle est le meilleur compromis pour ce qui compte ici : elle est facile à interroger, facile à expliquer à quelqu’un du métier, et c’est le modèle que les outils de BI attendent. Le flocon apparaît quand une dimension devient trop grosse pour être aplatie ; le Data Vault vit en amont,

comme couche d'intégration, et alimente ensuite une étoile ; la table large est ce qu'on construit en bout de chaîne pour un usage précis. Ces choix sont le cœur de la modélisation dimensionnelle, et ils appartiennent en propre au métier d'ingénieur analytique.

Deux versions de la même base — On travaille ici sur la version entrepôt (DW, dénormalisée en étoile). Il en existe une version OLTP, transactionnelle, découpée en 68 tables normalisées, qu'on charge dans PostgreSQL et qui sert de second terrain à partir du Cours 02. L'OLTP normalise pour écrire vite, l'entrepôt dénormalise en étoile pour analyser vite. Les deux sont mises en place au Cours 00.

Portabilité — Dans notre entrepôt, on écrit les noms dans la casse qu'on veut : `DimProduct` et `dimproduct` désignent la même table, parce que DuckDB compare les identifiants sans tenir compte de la casse tant qu'on n'utilise pas de guillemets. Il garde en revanche celle de la déclaration pour l'affichage, et c'est pourquoi une requête tapée en minuscules rend des en-têtes en `ProductKey`. Le portage PostgreSQL de la version OLTP, lui, replie tout en minuscules (`production.product`), et l'en-tête revient donc en minuscules.

6. Dans quel ordre une requête se lit

Labo : *Labo_01.md*, sous-partie 2.2.

Voici le point le plus utile du module. En SQL, vous décrivez ce que vous voulez, pas comment l'obtenir : le moteur se débrouille. Et il ne lit pas votre requête dans l'ordre où vous l'écrivez. Vous écrivez `SELECT` en premier ; il le prend en compte presque en dernier.

Voici cet ordre. Chaque étape reçoit les lignes de la précédente, comme un tapis roulant.

Vous écrivez	Le moteur prend en compte	Ce que l'étape fait
SELECT	1. FROM et JOIN	réunit les tables de départ
FROM	2. WHERE	garde ou écarte des lignes
WHERE	3. GROUP BY	regroupe en paquets
GROUP BY	4. HAVING	garde ou écarte des groupes
HAVING	5. SELECT	calcule les colonnes affichées
ORDER BY	6. ORDER BY	trie le résultat
LIMIT	7. LIMIT	coupe les N premières lignes

Un mot sur le nom qu'on donne à cet ordre, parce qu'il évite un contresens tenace. C'est un ordre logique, pas le déroulé des opérations dans la machine : il dit ce que la requête signifie, et il permet de prédire ce qu'elle renverra. Le moteur, lui, reste libre de s'y prendre autrement tant que le résultat est le même, et il ne s'en prive pas, par exemple en poussant un filtre à l'intérieur d'une jointure plutôt qu'après elle. L'ordre ci-dessus prédit donc le résultat, jamais le plan ni la vitesse : ceux-là s'observent, et c'est le Cours 10.

Cet ordre explique, à lui seul, des erreurs qui semblent absurdes :

Ce que vous constatez	Pourquoi
Un nom de colonne inventé dans le <code>SELECT</code> est « inconnu » dans le <code>WHERE</code> de PostgreSQL.	Le <code>WHERE</code> (étape 2) s'exécute avant le <code>SELECT</code> (étape 5). Le nom n'existe pas encore. DuckDB l'accepte quand même, par commodité (Cours 02, section 1).
<code>WHERE total > 100</code> refuse de fonctionner sur une somme.	Les sommes se calculent au <code>GROUP BY</code> (étape 3). Le <code>WHERE</code> est en amont : il filtre des lignes, pas des totaux. C'est le rôle du <code>HAVING</code> (étape 4).
Le même nom de colonne marche dans le <code>ORDER BY</code> .	Le <code>ORDER BY</code> (étape 6) vient après le <code>SELECT</code> (étape 5) : la colonne est déjà calculée.

La première ligne demande une précision, parce que c'est la seule des trois où nos deux terrains ne répondent pas pareil, et qu'un lecteur qui pratique d'abord l'entrepôt risque d'en tirer la mauvaise conclusion. Une requête qui reprend dans son `WHERE` un alias créé dans son `SELECT` passe sur DuckDB, qui résout l'alias avant de filtrer ; la même requête, sur la table équivalente de PostgreSQL, échoue sur « la colonne "ratio" n'existe pas ». C'est PostgreSQL qui suit la norme ici : l'alias naît au `SELECT`, à l'étape 5, et le `WHERE` est passé depuis l'étape 2. La commodité de DuckDB est donc un confort local, pas une règle du langage : le code qui s'en sert ne voyage pas.

À retenir — Ce tableau n'est pas à apprendre par cœur. Quand une requête refuse de partir, posez-vous une seule question : à cette étape-là, est-ce que cette colonne existe déjà ? Vous trouverez la cause tout seul.

7. Relier les tables : les jointures

Labo : Labo_01.md, sous-parties 2.2 et 2.3.

Une jointure combine les lignes de deux tables. Dans un entrepôt en étoile, on s'en sert sans arrêt : la table de faits ne contient que des clés et des mesures, les libellés lisibles vivent dans les dimensions. Les trois jointures à connaître :

Jointure	Elle garde...	Exemple sur AdventureWorks DW
<code>INNER JOIN</code>	seulement les lignes qui correspondent des deux côtés	les ventes rattachées à un produit connu
<code>LEFT JOIN</code>	chaque ligne de la table de gauche ; les colonnes de la table de droite valent <code>NULL</code> quand rien ne correspond	tous les produits, même sans sous-catégorie

Jointure	Elle garde...	Exemple sur AdventureWorks DW
SELF JOIN	une table jointe à elle-même	comparer deux lignes de la même table

Les mots « gauche » et « droite » prêtent à confusion, parce qu'ils ne parlent pas de la position d'une ligne dans une table : les lignes s'empilent de haut en bas, elles n'ont ni gauche ni droite. Ils parlent de l'ordre dans lequel les deux tables sont écrites dans la requête. Lisez `FROM dbo.DimProduct AS p LEFT JOIN dbo.DimProductSubcategory AS s` comme une phrase :

Table	Où elle est écrite	Son nom
DimProduct	dans le FROM, avant le mot JOIN	la table de gauche
DimProductSubcategory	après le mot JOIN	la table de droite

Le résultat, lui, se lit à l'horizontale. Le moteur prend une ligne de la table de gauche et colle, à sa droite, la ligne qui lui correspond dans la table de droite. Chaque ligne du résultat a donc deux moitiés : à gauche les colonnes venues de DimProduct, à droite celles venues de DimProductSubcategory. Trois lignes de ce résultat :

EnglishProductName, moitié gauche (DimProduct)	EnglishProductSubcategoryName, moitié droite (DimProductSubcategory)
Mountain-100 Black, 38	Mountain Bikes
Road-150 Red, 62	Road Bikes
Adjustable Race	NULL

La dernière ligne est tout le sujet. Adjustable Race est une pièce détachée sans sous-catégorie : sa moitié droite n'a rien à recevoir. Une jointure interne jetterait la ligne entière ; le `LEFT JOIN` la garde et remplit la moitié droite de NULL. Quand on dit qu'un `LEFT JOIN` « garde les lignes de la table de gauche », c'est cela : aucun produit ne disparaît du résultat. Sur les 606 lignes de DimProduct, 209 n'ont pas de sous-catégorie ; le `LEFT JOIN` rend les 606 lignes, une jointure interne n'en rendrait que 397.

À retenir — Gauche et droite, c'est l'ordre d'écriture : la table du FROM est à gauche, celle qui suit le JOIN est à droite. Le `LEFT JOIN` protège la table de gauche : chacune de ses lignes reste dans le résultat, et les colonnes de la table de droite valent NULL quand rien ne correspond. Le `RIGHT JOIN` existe et protège l'autre côté, mais on l'écrit rarement : on inverse plutôt l'ordre des deux tables pour revenir à un `LEFT JOIN`.

Le piège à connaître, c'est le `LEFT JOIN` avec une condition dans le WHERE. Le `LEFT JOIN` garde chaque ligne de la table de gauche, même quand la table de droite n'a rien à lui associer ; les colonnes de la table de droite valent alors NULL sur cette ligne, comme pour Adjustable Race. Mais si vous ajoutez une condition sur la table de droite dans le WHERE, elle rejette ces NULL, et votre `LEFT JOIN` se met à agir comme un `INNER JOIN`.

-- OBJECTIF : tous Les produits, avec Le nom de Leur sous-categorie s'ils en ont une.

```
-- FAUX : La condition sur s.EnglishProductSubcategoryName dans le WHERE fait
--          disparaître les produits sans sous-catégorie.
SELECT p.EnglishProductName, s.EnglishProductSubcategoryName AS sous_categorie
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s
      ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
WHERE s.EnglishProductSubcategoryName = 'Mountain Bikes';

-- JUSTE : La condition descend dans le ON, et chaque produit (table de gauche) reste.
SELECT p.EnglishProductName, s.EnglishProductSubcategoryName AS sous_categorie
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s
      ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
      AND s.EnglishProductSubcategoryName = 'Mountain Bikes';
```

Règle simple : une condition qui décrit comment relier les tables va dans le ON ; une condition qui filtre le résultat final va dans le WHERE. On approfondira les jointures au fil de la formation.

8. NULL : la valeur « on ne sait pas »

Labo : Labo_01.md, sous-partie 2.4.

NULL n'est pas zéro, ni une chaîne vide : c'est une valeur inconnue. Dans notre entrepôt, 211 lignes de DimProduct sur 606 n'ont pas de prix de liste : leur ListPrice vaut NULL. Ce sont surtout des pièces détachées qui ne se vendent pas seules, un roulement à billes, une lame. Cette inconnue change les règles du jeu. Toute comparaison avec NULL ne donne ni vrai ni faux, mais inconnu, et le WHERE ne garde que ce qui est franchement vrai.

Trois pièges reviennent tout le temps :

1. = NULL ne marche jamais. WHERE ListPrice = NULL ne renvoie aucune ligne, même quand le prix est bel et bien absent. La bonne façon de tester : WHERE ListPrice IS NULL.
2. Les totaux ignorent les NULL. COUNT(*) compte toutes les lignes ; COUNT(ListPrice) ne compte que celles qui ont un prix. AVG et SUM sautent aussi les NULL.
3. NOT IN se fait piéger par un NULL. Si la liste derrière NOT IN contient un seul NULL, la condition ne renvoie plus jamais rien. En cas de doute, on utilise NOT EXISTS.

Trois outils pour dompter NULL :

- COALESCE(a, b) renvoie a s'il existe, sinon b. Par exemple COALESCE(Size, 'sans taille').
- NULLIF(a, b) renvoie NULL si a égale b. Il sert à neutraliser une division par zéro, x / NULLIF(y, 0), et à démasquer une fausse valeur, comme on le verra à la section 9.

- `IS NULL` et `IS NOT NULL` sont la seule bonne façon de tester la présence ou l'absence.

Le deuxième outil mérite un mot, parce que ce dont il vous protège n'est pas le même danger sur les deux terrains. `SELECT 10 / 0` lève une erreur sur PostgreSQL, « division par zéro », et la requête s'arrête net. Sur DuckDB, la même expression rend `inf`, l'infini, sans un mot : la requête réussit, la valeur entre dans la suite des calculs, et une somme ou une moyenne qui la rencontre vaut `inf` à son tour. Des deux comportements, le second est le plus coûteux, parce qu'il produit un résultat qu'on peut recopier dans un rapport. `NULLIF(y, 0)` les évite tous les deux d'un coup : la division donne `NULL`, et les règles de cette section s'appliquent.

Portabilité — `COALESCE` et `NULLIF` marchent partout. Pour tester l'égalité en traitant `NULL` comme une valeur normale, DuckDB et PostgreSQL offrent tous deux `a IS DISTINCT FROM b`, avec le même résultat sur nos deux terrains : `1 IS DISTINCT FROM NULL` vaut vrai et Snowflake a la même écriture, d'après sa documentation. C'est la forme de la norme, et c'est elle qu'il faut chercher d'abord dans la documentation d'un moteur inconnu : celui qui ne l'a pas a presque toujours un opérateur à lui, sous un autre nom.

9. « NA » : quand l'absence se déguise en valeur

Labo : Labo_01.md, sous-partie 2.5.

Regardez les couleurs de notre table de produits avec un `GROUP BY Color` : vous ne trouverez aucun `NULL`. Vous trouverez à la place une couleur qui n'en est pas une, `NA`, sur 254 lignes. Microsoft a déclaré la colonne `NOT NULL`, et chaque produit sans couleur y porte ce libellé. C'est ce qu'on appelle une **valeur sentinelle** : une vraie valeur, choisie par convention pour dire « rien ».

Ce n'est pas une maladresse. Le transactionnel range la même information autrement : dans `production.product`, 248 produits ont une couleur `NULL`, et aucun ne vaut « NA ». Le libellé a donc été posé en chemin, au moment d'alimenter l'entrepôt, et c'est une pratique courante de la modélisation dimensionnelle. Un rapport affiche « NA » en toutes lettres là où un `NULL` laisserait une case vide, et un filtre ou un regroupement le traite comme n'importe quelle autre catégorie.

Le prix à payer, c'est que le moteur ne sait plus qu'il s'agit d'une absence. Les règles de la section 8 cessent toutes de s'appliquer.

Ce que vous écrivez	Si l'absence est un <code>NULL</code>	Si l'absence est « NA »
<code>WHERE Color IS NULL</code>	trouve les produits sans couleur	ne trouve rien, zéro ligne
<code>COUNT(Color)</code>	ne compte que les produits colorés	compte tout, « NA » compris
<code>GROUP BY Color</code>	un groupe <code>NULL</code> , facile à repérer	« NA » devient une couleur parmi d'autres, et la plus vendue en nombre de lignes
<code>COALESCE(Color, 'sans couleur')</code>	remplace l'absence	ne fait rien

Deux dangers s'ajoutent. D'abord, « NA » mélange des cas qui n'ont rien à voir : 56 lignes sont des produits du catalogue pour lesquels la couleur n'a pas de sens (une fourche, un guidon, une pompe), 198 sont des pièces détachées sans prix ni sous-catégorie. Un libellé donne l'illusion d'une information là où il n'y en a pas. Ensuite, la convention n'est pas uniforme, même dans une seule table : `SizeRange` écrit « NA » quand un produit n'a pas de taille, alors que `Size`,

ProductLine et Style restent à NULL. Et le transactionnel fait parfois l'inverse de l'entrepôt : le prix de ses 200 pièces détachées y vaut 0, là où l'entrepôt met NULL.

L'outil qui répare, c'est NULLIF. NULLIF(Color, 'NA') rend NULL partout où la colonne vaut « NA », et les règles de la section 8 s'appliquent de nouveau : COUNT(NULLIF(Color, 'NA')) donne les 352 lignes vraiment colorées. COALESCE fait le chemin inverse, et c'est d'ailleurs exactement ce que l'alimentation a fait : COALESCE(Color, 'NA').

À retenir — Devant une colonne que vous ne connaissez pas, regardez ses valeurs distinctes avant de compter ses absences. Un IS NULL qui répond zéro ne prouve pas que rien ne manque ; il prouve seulement que l'absence n'est pas écrite NULL. Cherchez les « NA », « N/A », « Unknown », les chaînes vides, et les 0 ou les -1 qui jouent le même rôle.

Piège — Une sentinelle dans une colonne numérique est bien plus dangereuse qu'un libellé, parce qu'elle entre dans les calculs sans rien dire. Côté transactionnel, les 200 pièces détachées ont un listprice à 0 : la moyenne des prix y tombe à 438,67 \$, contre 727,26 \$ quand on les écarte avec NULLIF(listprice, 0). Sur une mesure, l'absence s'écrit NULL.

Et maintenant : la pratique

La théorie s'arrête ici. Place à la manipulation : le laboratoire de ce cours (Labo_01.md) reprend chacune de ces idées sous forme d'étapes à réaliser sur la vraie base. L'installation, elle, est déjà faite au Cours 00.

Points clés à retenir

- Cinq mots : la *data* (les faits) forme une *database* (des données rangées), gérée par un *DBMS* (le logiciel) ; un *RDBMS* range tout en tables reliées (DuckDB, PostgreSQL, MySQL, Oracle) ; *SQL* est le langage qu'on lui parle.
- Cinq familles de commandes : DDL (structure, annulable sur nos deux terrains, validé d'office ailleurs), DML (données, annulable avant COMMIT), DQL (lire, SELECT), DCL (droits, absent de DuckDB), TCL (valider ou annuler). Ce cours travaille surtout le DQL.
- DuckDB (analytique, local, intégré) est le moteur de référence de la formation ; PostgreSQL (transactionnel, local, serveur) est le second terrain, à partir du Cours 02, où chaque module a deux laboratoires. Le SQL de base est le même partout, les détails changent (encadrés « Portabilité », qui nomment Snowflake d'après sa documentation).
- AdventureWorks DW est notre base fil rouge : un entrepôt en étoile (une table de faits FactInternetSales, des dimensions DimProduct, DimDate), tout dans le schéma dbo. On écrit schéma point table, par exemple dbo.DimProduct.
- L'étoile n'est pas le seul modèle : le flocon normalise les dimensions, le 3NF garde l'intégrité au prix des jointures, la table large aplatit tout, le Data Vault historise en amont. L'étoile domine parce qu'elle est le meilleur compromis entre lisibilité et performance de lecture.
- Clé primaire égale identifiant unique d'une ligne (souvent une clé de substitution dans un entrepôt) ; clé étrangère égale lien du fait vers une dimension. Et sans ORDER BY, l'ordre des lignes n'est pas garanti.

- L'ordre logique d'exécution (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT) explique les erreurs de noms de colonnes et le partage entre WHERE et HAVING.
 - Gauche et droite, c'est l'ordre d'écriture : la table du FROM est à gauche, celle qui suit le JOIN à droite. Le LEFT JOIN garde chaque ligne de la table de gauche ; une condition sur la table de droite placée dans le WHERE peut le dégrader en INNER JOIN.
 - NULL égale inconnu : on teste avec IS NULL, les totaux l'ignorent, NOT IN s'y fait piéger, et COALESCE sert à le remplacer.
 - « NA » n'est pas NULL : c'est une sentinelle, une vraie valeur qui dit « rien ». Le moteur la compte et la regroupe comme les autres ; NULLIF(Color, 'NA') la reconvertit en NULL. Avant de compter des absences, on regarde les valeurs distinctes.
-

Exercices

Les exercices de ce cours vivent dans Exercices_01.md, et leur corrigé dans Corrige_01.md. Faites-les après le laboratoire : les énoncés pratiques se tapent sur l'entrepôt, et plusieurs questions supposent que vous ayez vu de vos yeux le piège du LEFT JOIN et celui de = NULL.